

1. Indicar el **party_id**

Solo cambia este valor:

```
library(sparklyr)
library(dplyr)
```

```
party_id_target <- "AQUI_TU_PARTY_ID"
```

```
# Evita problemas si el ID contiene un apostrofe
party_id_sql <- gsub("'", "", party_id_target, fixed = TRUE)
```

Ahora recuperamos su **observation_date** y **label_3m** directamente del dataset final:

```
sdf_sql(
  sc,
  sprintf(
    "
    CREATE OR REPLACE TEMP VIEW selected_party_case AS

    SELECT
      party_id,
      observation_date,
      observation_year,
      label_3m

    FROM bu_frd.model_features_ab_final_2022_2025

    WHERE party_id = '%s'
    ",
    party_id_sql
  )
)
```

Primero verifica:

```
selected_party <- sdf_sql(
  sc,
  "
  SELECT *
  FROM selected_party_case
  "
) %>%
  collect()

selected_party
```

Para tu caso deberías ver algo parecido a:

party_id	observation_date	observation_year	label_3m
XXXXXXX	2024-xx-xx	2024	1

2. Todos los rows originales de **trades** desde 2022

Este es el pull histórico principal que estabas buscando.

No tiene **LIMIT**. No tiene **DISTINCT**. No tiene **ROW_NUMBER**. No elimina filas.

```
party_history_raw <- sdf_sql(
  sc,
  "
  WITH raw_history AS (

    SELECT
      s.observation_date,
      s.observation_year,
      s.label_3m,

      last_day(t.data_date) AS history_month,

      CAST(
        months_between(
          last_day(s.observation_date),
          last_day(t.data_date)
        ) AS INT
      ) AS history_month_offset,

      t.*

    FROM trades t

    INNER JOIN selected_party_case s
      ON t.party_id = s.party_id

    WHERE t.data_date >= to_date('2022-01-01')

    /* Nothing after the observation date */
    AND t.data_date <= s.observation_date

  )
```

```

SELECT
  *,

CASE

  /* Older than the official feature window */
  WHEN history_month_offset > 12
    THEN 'HISTORICAL'

  /* Technical month used for first MOP transition */
  WHEN history_month_offset = 12
    THEN 'SEED'

  /* 9-month baseline */
  WHEN history_month_offset BETWEEN 3 AND 11
    THEN 'PRIOR_9M'

  /* Most recent 3 months */
  WHEN history_month_offset BETWEEN 0 AND 2
    THEN 'RECENT_3M'

  ELSE 'CHECK_DATE'

END AS period_type

FROM raw_history

ORDER BY
  data_date,
  acct_id,
  trades_seq_num
"
)

```

Si quieres traer **todos los rows a R**:

```

party_history_raw_df <- party_history_raw %>%
  collect()

```

```

party_history_raw_df

```

Con un solo **party_id**, normalmente debería ser manejable.

Esta tabla contendrá literalmente todas las columnas originales que mencionaste:

```

party_id
trades_seq_num
acct_id

```

acct_num
acct_typ_cde
agcy_id_cde
...
acct_mop_cde
cr_lmt_amt
curr_bal_amt
past_due_amt
high_cr_amt
eff_dte
open_dte
...
data_date

más las columnas que agregamos:

observation_date
observation_year
label_3m
history_month
history_month_offset
period_type

Por ejemplo, podrías terminar viendo algo así:

history_m onth	period_ty pe	acct _id	acct_typ _cde	acct_mop _cde	curr_bal_ amt	past_due _amt	data_d ate
2022-01-3 1	HISTORI CAL	A00 1	CC	01	2,350	0	2022-0 1-31
2022-01-3 1	HISTORI CAL	A00 2	AU	01	9,100	0	2022-0 1-31
2022-02-2 8	HISTORI CAL	A00 1	CC	01	2,510	0	2022-0 2-28
...	...	...	...	...	...	...	...
2023-12-3 1	SEED	A00 1	CC	01	4,100	0	2023-1 2-31
2024-01-3 1	PRIOR_9 M	A00 1	CC	01	4,250	0	2024-0 1-31
...	...	...	...	...	...	...	...
2024-11-3 0	RECENT _3M	A00 1	CC	02	5,100	125	2024-1 1-30

2024-12-31	RECENT_3M	A001	CC	01	4,800	0	2024-12-31
------------	-----------	------	----	----	-------	---	------------

Los números del ejemplo son ilustrativos.

3. Variables mensuales que usamos para crear las features

Ahora sacamos las variables que ya fueron construidas oficialmente por el pipeline.

Estas **solo existen en los 12 meses oficiales**, porque esa tabla fue diseñada precisamente para **PRIOR_9M + RECENT_3M**.

```
party_monthly_behavior <- sdf_sql(
  sc,
```

```
  "
  SELECT
```

```
    m.party_id,
    m.observation_date,
    m.label_3m,
```

```

    m.feature_month,
    m.month_offset,
    m.feature_window,
```

```

    /* Account exposure */
    m.reported_accounts,
    m.active_accounts,
    m.rated_active_accounts,
    m.utilization_eligible_accounts,
```

```

    /* Balance */
    m.avg_balance_active,
    m.total_balance_active,
```

```

    /* Past due */
    m.avg_past_due_active,
    m.total_past_due_active,
```

```

    /* Delinquency */
    m.delinquent_active_accounts,
    m.delinquent_account_share,
```

```

/* Utilization */
m.avg_utilization,

/* MOP-based monthly behavior */
m.deterioration_count,
m.improvement_count,

m.adverse_entry_count,
m.adverse_resolved_count,

m.became_unrated_count,
m.left_unrated_count,

m.max_dpd_jump

FROM bu_frd.model_party_behavior_monthly_2022_2025 m

INNER JOIN selected_party_case s
  ON m.party_id = s.party_id
  AND m.observation_date = s.observation_date

ORDER BY
  m.month_offset DESC
"
) %>%
collect()

party_monthly_behavior

```

Aquí deberían salir exactamente:

12 rows

correspondientes a:

```

M-11 ... M-3  PRIOR
M-2 ... M0    RECENT

```

4. Agregar weighted utilization mensual

Como weighted utilization quedó construido en otra tabla:

```

party_monthly_utilization <- sdf_sql(
  sc,
  "

```

```

SELECT
  u.*

FROM bu_frd.model_party_utilization_monthly_2022_2025 u

INNER JOIN selected_party_case s
  ON u.party_id = s.party_id
  AND u.observation_date = s.observation_date

ORDER BY
  u.month_offset DESC
"
) %>%
collect()

party_monthly_utilization

```

Aquí puedes ver exactamente las variables mensuales utilizadas para:

```

avg_utilization_change_model
weighted_utilization_change_model
avg_utilization_z_model
weighted_utilization_z_model

```

5. Los 13 meses oficiales con MOP transitions

Para entender las variables como:

```

deterioration_count
adverse_entry_count
improvement_count
became_unrated_count

```

conviene conservar el pull de transiciones.

Y aquí sí aparecen los:

13 meses

porque necesitamos **SEED**.

```

party_mop_transitions <- sdf_sql(
  sc,

```

```
"
SELECT
  t.*

FROM bu_frd.model_trades_transition_13m_2022_2025 t

INNER JOIN selected_party_case s
  ON t.party_id = s.party_id
  AND t.observation_date = s.observation_date

ORDER BY
  t.month_offset DESC,
  t.acct_id
"
)
```

Y si quieres todos los rows:

```
party_mop_transitions_df <- party_mop_transitions %>%
  collect()
```

```
party_mop_transitions_df
```

Aquí tampoco hay **LIMIT**.

6. Prior 9M vs Recent 3M

Esta tabla es el puente entre las variables mensuales y las engineered features.

```
party_behavior_windows <- sdf_sql(
  sc,
  "
SELECT
  w.*

FROM bu_frd.model_party_behavior_windows_2022_2025 w

INNER JOIN selected_party_case s
  ON w.party_id = s.party_id
  AND w.observation_date = s.observation_date
"
) %>%
  collect()

party_behavior_windows
```


Aquí es donde conceptualmente puedes seguir:

12 monthly values

hacia:

Prior9 aggregate

y:

Recent3 aggregate

7. Fila final con todas las engineered features

Finalmente recuperamos la fila exacta del dataset final:

```
party_final_features <- sdf_sql(
  sc,
  "
  SELECT
    f.party_id,
    f.observation_date,
    f.observation_year,
    f.label_3m,

    /* Transition behavior */
    f.deterioration_acceleration,
    f.adverse_acceleration,
    f.max_dpd_jump_recent_3m,

    /* Active accounts */
    f.active_accounts_change_vs_baseline,
    f.active_accounts_changed_flag,

    /* Balance */
    f.balance_change_model,
    f.log1p_abs_balance_change_model,
    f.balance_change_available_flag,
    f.balance_recent_months_available,
    f.balance_prior_months_available,
```

```

/* Past due */
f.past_due_change_model,
f.past_due_change_available_flag,
f.past_due_recent_months_available,
f.past_due_prior_months_available,

/* Delinquency */
f.delinquency_share_change_model,
f.delinquency_change_available_flag,
f.delinquency_recent_months_available,
f.delinquency_prior_months_available,

/* Utilization */
f.avg_utilization_change_model,
f.weighted_utilization_change_model,
f.utilization_change_available_flag,
f.utilization_recent_months_available,
f.utilization_prior_months_available,

/* Z challengers */
f.active_accounts_z_model,
f.active_accounts_z_available_flag,

f.avg_utilization_z_model,
f.avg_utilization_z_available_flag,

f.weighted_utilization_z_model,
f.weighted_utilization_z_available_flag

FROM bu_frd.model_features_ab_final_2022_2025 f

INNER JOIN selected_party_case s
  ON f.party_id = s.party_id
  AND f.observation_date = s.observation_date
"
) %>%
collect()

party_final_features

```

Aquí debería aparecer **una sola fila**.

Entonces tendrás exactamente esta estructura

Tu primer objeto:

party_history_raw_df

será el que te permite literalmente explorar:

2022 → 2023 → 2024 → *ObservationDate*

con **todos los registros originales de trades**.

Luego:

party_monthly_behavior

te muestra las variables mensuales oficiales utilizadas en los 12 meses de features.

Después:

party_mop_transitions_df

te muestra los 13 meses de transitions, incluyendo **SEED**.

Y finalmente:

party_final_features

te muestra la única fila donde existen:

Prior9 vs Recent3

y, por tanto, las features finales.

La ventaja de esta estructura es que **no estamos alterando ni reinterpretando la historia anterior a la ventana oficial**. Simplemente la estamos mostrando completa, exactamente como querías, mientras conservamos por separado las variables creadas oficialmente por el pipeline.